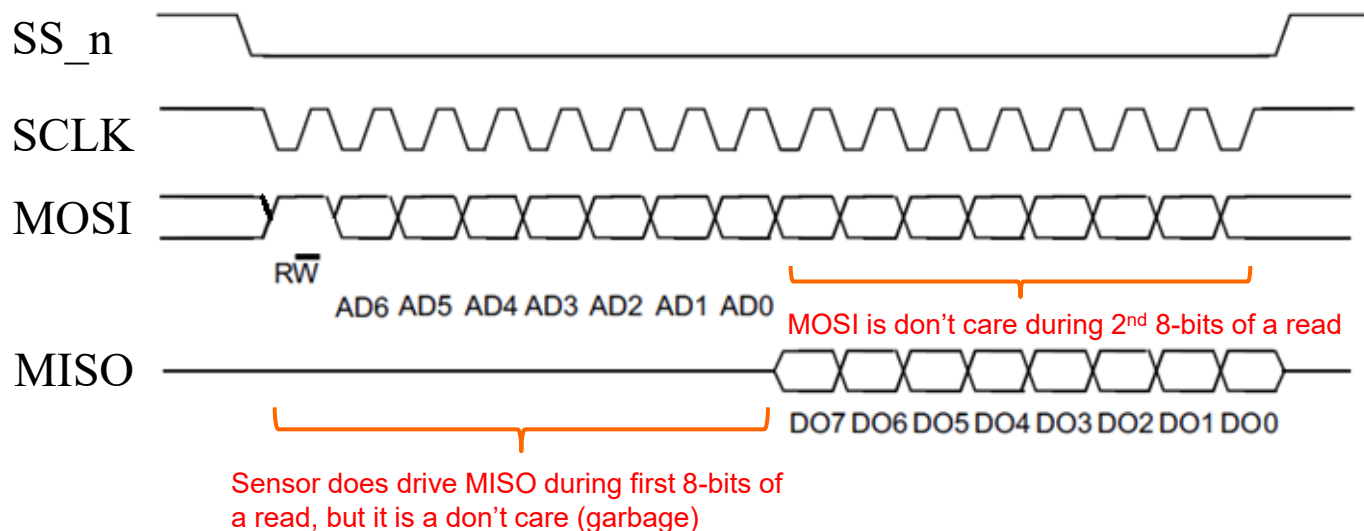




SPI Transceiver Testing

- The inertial sensor (Nemo family from ST micro) is configured via a SPI interface
- You can think of a 16-bit transaction as two 8-bit transactions
 - First 8-bits → sensor is looking at MOSI to see what register is being read/written. The MSB is a $R/\bar{W}$ bit, the next 7 bits is the address of the register.
 - Second 8 bits → If it is a read, the data at the requested register will be returned on MISO.

SPI Read waveform

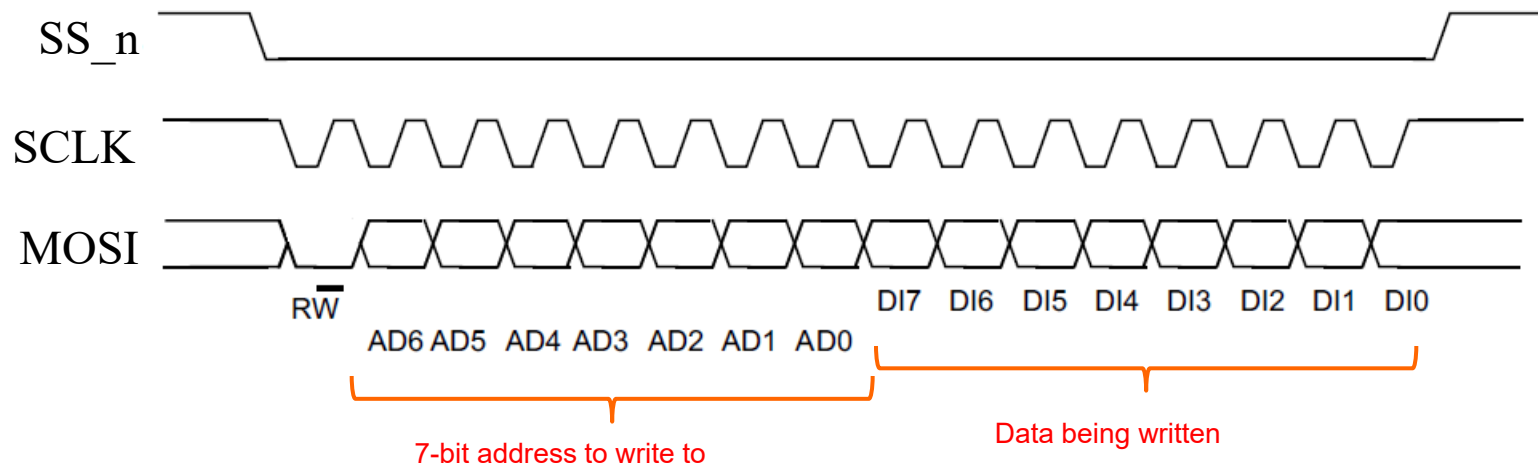




SPI Transceiver Testing

- During a write to inertial sensor:
 - First 8 bits → specify it is a write and the address to write to
 - Second 8 bits → specify the data being written
 - Data returned on MISO is ignored

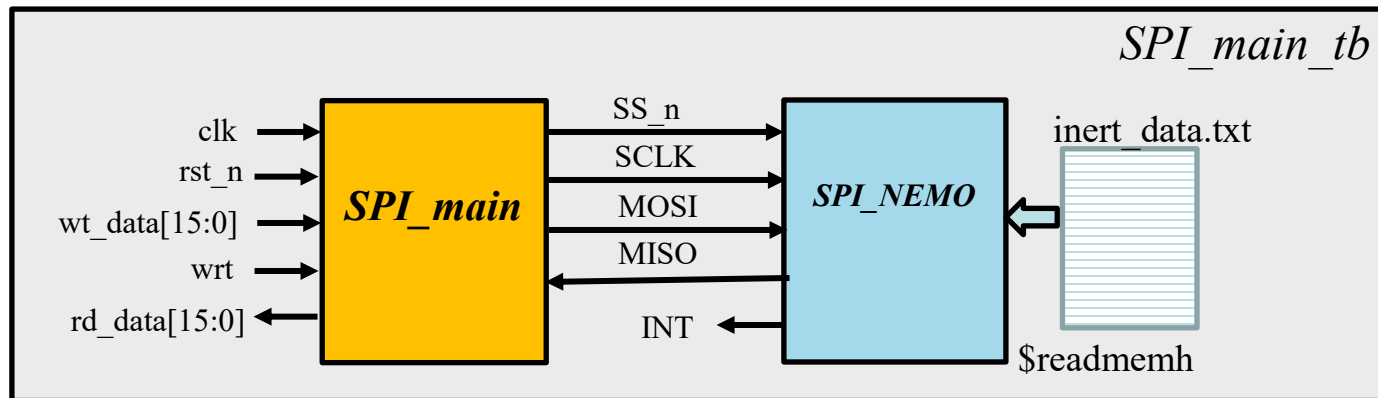
SPI Write waveform





SPI Transceiver Testing

- We provide a model of ST's Nemo inertial sensor for you to interface with
 - Download **SPI_NEMO.sv** from Canvas
- Create a self-checking testbench **SPI_main_tb.sv** and submit it



- This model reads off a text file containing some stored inertial data
 - Each row in *inert_data.txt* represents a set of valid readings (AX, AY, ... roll, yaw)
- It will simulate what happens in an inertial sensor
 - A new set of valid data will be available every fixed interval
 - We can configure it to send an interrupt signal (INT) whenever a new reading is available
- The model also has a "WHO_AM_I" register for identification



SPI Transceiver Testing

Registers to read from

Addr	Description
7'h0F	Who Am I register (returns 0x6A)
7'h26	yawL → yaw rate low from gyro
7'h27	yawH → yaw rate high from gyro

Register to write to

Addr	Description
7'h0D	INT config register. Writing 0x02 to it will configure it to assert INT when new data is ready.

In your testbench, you should do the following:

- **Read** from the **WHO_AM_I** register & confirm we received 0x6A
→ `set wt_data = 16'h8Fxx, set wrt, expect rd_data = 16'hxx6A`
- **Write** 0x02 to **INT** register to configure interrupt
→ `set wt_data = 16'h0D02, set wrt`
- Whenever **ptchL** is read, INT will be cleared
- INT will be asserted again when next reading is available (i.e., next row in `inert_data.hex`)
- For your testbench, you should read a few more registers to ensure everything is working correctly
→ Look at `inert_data.hex` to figure out the expected values